JNEOPALLIUM · AD-FRAUD GUARDIAN
ARCHITECTURE ARTICLE

Stop Paying for Traffic That Never Happened

Multi-label invalid-traffic detection — complete architecture, explained for everyone

---

How a brain-inspired engine reads every ad event across many timescales, names the exact kind of invalid traffic, shows its evidence, and recommends an action — without ever silently blocking a publisher or withholding a payout. Written so a non-specialist can follow every step.

Document	Architecture & Technical Overview
Product	Jneopallium Ad-Fraud Guardian (advertising-fraud module)
Model	advertising-fraud 1.0.0 · macro-F1 ≈ 0.97
Safety mode	ADVISORY / SHADOW (recommend-only)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
Audience	Marketers, ad-ops, anti-fraud teams, engineers, newcomers
License	BSD 3-Clause

What's inside

1 Executive summary

A large share of digital advertising is paid for, but never seen by a real interested human. Bots click ads, click farms manufacture installs, fraudsters hijack the credit for conversions that would have happened anyway, and "made-for-advertising" inventory soaks up budget. Industry estimates put the global loss in the **tens of billions of dollars a year**.

The **Jneopallium Ad-Fraud Guardian** scores every advertising event — a bid, an impression, a click, an install, a postback — and answers three questions a single "fraud score" cannot: **which** kind of invalid traffic this is, **why** (the evidence), and **what to do** (a proportionate, candidate action). It is **multi-label**: one event can be flagged as several things at once, each with its own calibrated probability.

> **The one sentence to remember**
>
> It does not output a vague "risk %." It says "this looks like click injection and attribution hijack, here is the evidence, recommend HOLD_PAYOUT for review" — and it never executes that action on its own.

Crucially, it runs **advisory-first** (SHADOW then ADVISORY): it recommends, it never auto-blocks a publisher, withholds a payout, or accuses a person. It is private by design — identifiers are hashed before training, and raw IPs, precise location, and user-agent strings are never exported. That makes it safe to run next to your existing measurement and billing.

2 The problem: where the ad budget leaks

A single "fraud / not-fraud" verdict is not enough

Invalid traffic is not one thing. It is a family of very different behaviours, each needing a different response and a different owner:

- **Bots** — automated clicks and views with no human behind them.
- **Click farms & incentivized installs** — real but motivated humans paid to click or install, who never become real customers.
- **Attribution fraud** — click spam and click injection that steal the *credit* for a conversion that would have happened anyway, so the wrong partner gets paid.
- **Event spoofing** — forged or replayed impressions, clicks, and postbacks.
- **Inventory / supply-path spoofing** — a publisher pretending to be premium inventory it is not (ads.txt / sellers.json mismatches).

A tool that returns one number — "73% fraud" — cannot tell an analyst whether to withhold a payout (attribution fraud), discount the traffic (low value), rate-limit a source (bots), or do nothing (a genuine accidental click). The expensive mistakes come from treating all of these as the same thing.

Why this is hard, honestly

The evidence for these families lives on **different timescales**: a forged signature is instant, a click-injection timing anomaly is per-session, a click-farm pattern emerges over minutes, and the truth about *quality* — did the install ever become a retained, paying user? — only arrives days later as retention, refund, and chargeback data. No single fast threshold can see across all of that.

3 The big idea: name the fraud, show the evidence

The Guardian borrows its shape from the brain: many specialised "neurons" running at different clock speeds, each producing one kind of evidence, feeding a final correlator that fuses everything into named, calibrated verdicts. Three design choices make it commercially credible:

- **Multi-label, not one score.** Ten independent fraud labels, each with its own calibrated probability, so the right action attaches to the right problem.
- **Evidence-first.** Every verdict carries the contributing signals and a human-readable reason, so an analyst — or an auditor, or a publisher appeal — can see *why*.
- **Advisory by construction.** It produces *candidate* actions only. Withholding money or blocking a partner requires first-party validation, legal review, and an appeal path — never the model alone.

> **Why advisory-first is the selling point, not a limitation**
>
> Wrongly withholding a legitimate publisher's payout is a lawsuit and a broken partnership. A system that recommends and explains — and lets a human pull the trigger — is the only kind a network can actually deploy at scale.

4 The platform underneath: the Jneopallium framework

The Guardian is one application of **Jneopallium**, a Java framework for biologically-grounded neuron networks (published in the *International Journal of Science and Research*, 2024). Three of its ideas explain why it suits this job.

1. Typed signals — meaning, not just numbers

A bid, a click, a postback, and a refund are *different kinds of signal* with different meaning and urgency. The engine routes each to the right specialist and preserves the evidence all the way to the verdict — which is what lets every advisory carry a reason.

2. Many clocks — instant checks, slow truth

Jneopallium processes fast signals every tick and slower context every N ticks. Integrity checks fire instantly; behavioural aggregation is slower; the delayed truth about traffic quality (retention, refunds, chargebacks) arrives slowest of all. One neuron can weigh an instant signature failure against a multi-day quality outcome in a single coherent judgement.

3. Single-purpose, swappable neurons

Each neuron does one job and can be upgraded or replaced for a particular deployment without touching the rest. Capacity is added by adding **layers and neurons**, not by stuffing logic into one place — which is exactly how this model was recently extended (see Section 10).

5 Architecture overview: the eight-layer network

Events flow one way through increasingly intelligent layers; an explainable advisory flows out the other end. The deployable network is **8 layers of 22 neurons over 21 features**:

Layer	Size	Job, in plain language

0 — Input	—	Receive multi-source ad events (bids, impressions, clicks, postbacks…)
1 — Event integrity	4	Signature/replay/clock integrity, human-interaction, session order, attribution
2 — Entity / graph / quality	3	Publisher baselines, click-farm graph fanout, delayed traffic quality
3 — Behavioural evidence	4	Family-specific evidence that separates the look-alike fraud types
4 — Feature interaction	8	Non-linear combinations of the evidence (one neuron per unit)
5 — Trained correlation	1	Fuses everything into ten calibrated label probabilities
6 — Response gate	1	Maps probabilities to a proportionate candidate action (advisory)
7 — Result	1	Emit the auditable decision: labels, evidence, reason, action

6 What it sees: the canonical ad event

Everything is normalised into one versioned **canonical event** that keeps both event time and ingestion time, and supports fifteen event types across the full funnel:

```
BID_REQUEST · IMPRESSION · VIEWABLE_IMPRESSION · CLICK · LANDING
INTERACTION · INSTALL · REGISTRATION · PURCHASE · POSTBACK
REFUND · CHARGEBACK · UNINSTALL · RETENTION · PAYOUT
```

Each event carries integrity fields (signatures, nonces, attestation), behavioural evidence (dwell, pointer dynamics, session counts), supply-chain context (ads.txt / sellers.json), and **delayed quality outcomes** (retention, refund, chargeback). Identifiers are pseudonymous: raw personal identifiers are HMAC-hashed before the model ever sees them, and a missing value is kept distinct from a zero-risk value.

7 The ten fraud families it scores

Label	Plain-language meaning
bot	Automated, non-human clicks or views
incentivized	Real but motivated users paid to act; low downstream value
clickFarm	Coordinated human/device farms manufacturing engagement
eventSpoofing	Forged or replayed impressions / clicks / postbacks
clickSpam	Mass low-quality clicks claiming broad attribution credit
clickInjection	A click fired at install time to steal attribution

attributionHijack	Taking credit for a conversion that would have happened anyway
inventorySpoofing	Publisher / supply-path misrepresentation (ads.txt / sellers.json)
accidentalOrLowValue	Genuine but accidental or low-value traffic — not malicious
unknownSuspicious	Anomalous traffic the model cannot yet name

Note `accidentalOrLowValue` and `unknownSuspicious`: the model is built to say *"this is odd but not necessarily fraud"* rather than over-accuse. That distinction is what keeps it usable without burning publisher relationships.

8 Signals on many clocks

Each signal family declares how often it is processed — the "many clocks" idea made concrete. Fast deterministic evidence runs every tick; the truth about quality arrives slowest:

Signal family	Cadence (loop / epoch)	What it captures
Event integrity, bid, impression, click, postback	1 / 1	Fast deterministic evidence
User-interaction aggregation	1 / 2	Behavioural aggregation
Session sequence & attribution	1 / 5	Causal-chain checks
Entity baseline	2 / 1	Adaptive per-publisher baselines
Graph cluster update	2 / 3	Rolling click-farm fanout graph
Traffic quality	2 / 5	Delayed quality evidence
Retention / refund / billing	2 / 10	Slow delayed truth
Model drift	2 / 60	Monitoring cadence

The physical duration of a tick is deployment configuration; these are the *semantic* scheduling cadences that let one model react instantly yet still incorporate evidence that only becomes available days later.

9 From raw event to verdict: the evidence pipeline

Walk one event through the network:

1. **Integrity receptors (Layer 1)** check the signature, nonce, client/server agreement, and session ordering, and produce fast risk evidence.
2. **Entity, graph and quality (Layer 2)** compare the event to per-publisher baselines, update a rolling device/account fanout graph, and fold in delayed retention/refund evidence.
3. **Behavioural-evidence neurons (Layer 3)** each emit one family-specific signal — click volume (spam), conversion timing (injection), incentive pattern (incentivized), low-value quality — which is what separates fraud types that otherwise look identical.
4. **Feature-interaction neurons (Layer 4)** form non-linear combinations of the evidence, so the final stage can express rules like "high attribution *and* low quality *and* high click rate."

5. **The correlation neuron (Layer 5)** fuses all of it into ten **calibrated** probabilities — one per fraud label.
6. **The response gate (Layer 6)** maps those probabilities to a proportionate **candidate** action, and the **result layer (Layer 7)** emits the auditable decision with its evidence and reason.

> **Response bands → candidate actions**
>
> log (0.0–0.30) → ALLOW/LOG · review (0.30–0.60) → REVIEW · rate-limit candidate (0.60–0.75) · hold-payout candidate (0.75–0.90) · escalate-to-analyst (0.90–1.0). Every action is a *candidate*; none is executed automatically.

10 The trained model and the deployable network

Training does not stop at a set of weights — it emits the **actual deployable JNeopallium network**: the eight layer files plus a descriptor, with the trained, calibrated heads embedded and referencing concrete runtime classes. The current network reflects a recent, deliberate upgrade that lifted quality sharply (see the Test Report):

Component	What it adds
21 features	8 base risks + 5 behavioural-evidence features + 8 non-linear interaction units
Behavioural-evidence layer	Single-purpose neurons that de-conflate look-alike fraud families
Feature-interaction layer	A non-linear hidden layer (added as its own layer, one neuron per unit)
Fitted, calibrated heads	Class-weighted logistic heads with real per-label Platt calibration
Cost-aware thresholds	Operating points chosen by expected utility (false positives cost money)

Because the trained heads, calibration, and thresholds all travel inside the network files, **what was trained is exactly what runs** — there is no translation step between the model and the deployed scorer.

11 Safety and privacy by design

1. **Advisory / shadow by default.** The response gate produces candidate actions only; automatic billing rejection, payout withholding, account blocking, or accusations are structurally disabled until a separate validation, legal-review, and appeal process exists.
2. **No accusation of a person.** The model emits invalid-traffic *evidence*; it never asserts that a named individual intentionally committed fraud.
3. **Privacy-preserving.** Device and account identifiers are pseudonymised (HMAC) before training; raw IPs, precise geolocation, and user-agent strings are never exported; a missing value is kept distinct from a zero-risk value.
4. **Leakage-safe evaluation.** Data is split by scenario, campaign episode, and deterministic replicate block — never by random row — with an adversarial holdout, so reported quality is not inflated by near-duplicates.

12 How good is it? (and how honest)

On the bundled reference corpus the model reaches **macro-F1 ≈ 0.97** across the ten labels, with most families at or near perfect separation and calibrated probabilities. A recent upgrade raised macro-F1 from **0.66 to 0.97** by adding the behavioural-evidence and interaction layers and fixing calibration and thresholds.

> **Read this honestly**
>
> That score is on a deterministic simulator plus weak public-source metadata, evaluated leakage-safe. It proves the pipeline and the label separation — it is NOT a claim of real-world accuracy, which must be earned on first-party labelled production traffic. The model stays SHADOW/ADVISORY until then. The Test Report states this plainly.

13 Where it fits in your stack

The Guardian sits *beside* your existing pipeline as an advisory scorer, not in the critical serving path:

```
ad server / SSP / DSP / MMP  -> canonical ad events
   -> Ad-Fraud Guardian (scores every event, advisory)
         -> decisions: labels + evidence + candidate action
   -> your SIEM / data warehouse / billing review / analyst queue
```

It deploys local, as an HTTP scoring service, or distributed; it reads a Kafka-style event stream or a direct feed; and it runs in three deployment shapes (single process, HTTP cluster, gRPC) so it fits a pilot on a laptop and an enterprise event firehose alike. Because correlation is by event time, delayed and out-of-order events still land in the right place.

14 Glossary for non-specialists

Term	Plain-language meaning
Invalid traffic (IVT)	Ad events with no genuine interested human behind them
Multi-label	One event can carry several independent fraud labels at once
Attribution	Deciding which click/partner gets credit (and payment) for a conversion
Click injection	Firing a fake click at install time to steal that credit
Postback	A server-to-server message confirming a conversion happened
ads.txt / sellers.json	Public files declaring who is authorised to sell an ad slot
Calibration	Making a 0.9 score really mean ~90% likely
Advisory / shadow	The system recommends; humans decide. It never blocks or withholds on its own
Pseudonymous / HMAC	Identifiers are hashed so the model never sees raw personal data

Macro-F1	The detection quality averaged equally across all ten fraud labels

Jneopallium Ad-Fraud Guardian · advertising-fraud module. Architecture article for technical and non-technical readers. Safety mode: ADVISORY / SHADOW. License: BSD 3-Clause.